# Impacto de la latencia en el agendamiento de MedConnect

Relación entre el diseño de la API y la cadena de valor del aliado

Una agregación síncrona ineficiente puede impedir que MedConnect confirme citas en 3 segundos y reducir la conversión del centro médico aliado. La demora técnica se traslada al punto donde una intención del paciente debe convertirse en una reserva efectiva. El daño potencial incluye abandono, trabajo manual y menor aprovechamiento de la agenda. El modelo permite rastrear esa dependencia desde la persistencia de los datos hasta el objetivo de negocio; su magnitud debe comprobarse con mediciones.

## Sustento en los elementos de Motivación

El **Driver «Demanda de agendamiento digital ágil»** expresa la necesidad de reducir fricción en el acceso a citas. Mediante **Influence** orienta el **Goal «Aumentar conversión a citas confirmadas»**. El **Outcome «Agendamiento confirmado en máximo 3 s»** realiza ese objetivo: aporta un resultado observable de rapidez, aunque la rapidez por sí sola no garantiza conversión. El proceso **«Agendar y confirmar cita»** realiza el Outcome y recibe un **Serving** del servicio de aplicación **«Consultar disponibilidad y confirmar cita»**. Así se conectan las capas arquitectónicas [1]: una decisión de implementación puede comprometer el resultado que sustenta la estrategia.

## Cómo surge el cuello de botella

La interfaz **«API_Riesgo_v1 (REST/JSON)»**, cuyo nombre conserva el requisito del taller, expone el servicio de agendamiento. El **«Motor de agendamiento MedConnect»** lo realiza y accede a **«Agenda y citas»**; el Artifact **«Base de datos de citas»** materializa esos datos en el nodo **«SQL Azure»**. Hay desacoplamiento contractual, pero el tiempo de respuesta sigue dependiendo del código, la red y la persistencia. La interfaz no elimina esas dependencias.

El antipatrón aparece si cada confirmación espera llamadas encadenadas para reunir agenda, datos del paciente y consultorio, incluyendo información prescindible, sin límites de tiempo ni control de reintentos. REST y la agregación síncrona no son por sí mismos antipatrones: la ineficiencia proviene de esa cadena crítica y de su acoplamiento temporal. Una dependencia lenta retiene la solicitud completa; los fallos pueden propagarse y los reintentos aumentar la carga [2].

**Ejemplo didáctico, no medición:** con 0,25 s de entrada, consultas independientes de 0,90, 1,00 y 0,80 s, y 0,35 s de reserva y respuesta, la ejecución serial suma **3,30 s**. Si las consultas se paralelizan de forma segura, la ruta ideal sería 0,25 + máx(0,90; 1,00; 0,80) + 0,35 = **1,60 s**. Son tiempos de una solicitud hipotética, no percentiles sumables ni una garantía bajo carga.

## Efecto sobre la cadena de valor del aliado

La cadena pasa de búsqueda y selección de horario a reserva confirmada, atención y eventual facturación. Si la API demora o vence, el paciente puede abandonar antes de reservar o acudir a otro canal. El aliado pierde oportunidades de ocupar horarios; recepción recibe consultas y debe verificar si la reserva se guardó. Si la escritura terminó después de que el cliente agotara su espera, un reintento sin idempotencia puede producir duplicados. La aparente solución manual añade costo operativo y puede degradar la experiencia del paciente.

Por tanto, la degradación del Application Service afecta el Business Process, pone en riesgo el Outcome de 3 segundos y debilita la realización del Goal de conversión. El Driver persiste aunque se publique una API: digitalizar una espera no resuelve la necesidad de agilidad. Más capacidad de ejecución en **«Kubernetes Cloud»** tampoco asegura la solución si SQL Azure conserva consultas lentas, bloqueos o un exceso de conexiones. Esta es una hipótesis arquitectónica que requiere trazas y pruebas, no un diagnóstico de infraestructura desplegada.

## Indicadores que hacen verificable el impacto

Defino **conversión = citas confirmadas / solicitudes válidas de agendamiento**, dentro de una misma ventana y cohorte, contando cada intención una vez para que los reintentos no alteren el denominador. Como ejemplo hipotético, pasar de 800 a 700 confirmaciones entre 1.000 solicitudes reduce la conversión del 80 % al 70 %: 10 puntos porcentuales y 100 reservas menos. No equivale automáticamente a 100 consultas perdidas ni a ingresos perdidos, porque pueden existir recuperación por otro canal, cancelaciones y ausencias.

La operacionalización propuesta del Outcome es **p95 del tiempo de confirmación extremo a extremo ≤ 3 s**, desde la acción de confirmar hasta recibir una reserva persistida. Debe observarse junto con p99, errores, timeouts, abandono y conversión; el promedio puede ocultar demoras y medir solo respuestas exitosas puede ocultar fallos. Para atribuir cambios al diseño se necesita una línea base, segmentar tráfico y demanda y comparar cohortes equivalentes o un experimento controlado. No existen mediciones de producción en este ejercicio.

## Decisión de arquitectura y criterio de aceptación

Recomiendo acotar las consultas a los datos indispensables, ejecutar en paralelo las lecturas independientes y fijar presupuestos de tiempo por dependencia. La notificación puede desacoplarse mediante el componente existente **«Webhook notificaciones»**; la confirmación debe esperar la reserva consistente. Se requieren idempotencia y control de concurrencia para evitar dobles citas. La caché puede acelerar la consulta, pero el horario debe validarse al reservar. Los límites de reintento y la protección ante fallos evitan amplificar una dependencia degradada [2].

La aceptación exige probar el recorrido completo bajo carga representativa y fallos de dependencias, verificar la meta de latencia sin aumentar errores ni duplicados y contrastar la conversión del aliado. No debe declararse éxito por devolver una respuesta rápida sin reserva confirmada. Así se evalúa la decisión técnica por su contribución al Outcome y al Goal, preservando la intención expresada por el Driver.

## Referencias

[1] The Open Group. ArchiMate 101 A Practical Introduction. Relaciones, dominios y capas.
archimate-community.pages.opengroup.org/workgroups/archimate-101/
[2] Microsoft Azure Architecture Center. Patrón de agregación de gateway.
learn.microsoft.com/es-es/azure/architecture/patterns/gateway-aggregation